

Gestión de Enjambres

Objetivo de la Práctica

En esta práctica aprenderán a configurar y gestionar un enjambre de UAVs Crazyflie empleando la librería oficial de Bitcraze. Comenzaremos estableciendo la comunicación radio con múltiples drones, creando y administrando un objeto Swarm que permita lanzar de forma simultánea comportamientos de vuelo. A continuación, implementaremos funciones de control que coordinen despegues, mantenimientos de posición y aterrizajes en paralelo, garantizando la sincronización y la seguridad entre los vehículos.

Parte 1: Vuelo Autónomo con enjambre

A continuación, se explica como mover varios UAV a una posición determinada a través de un script en python.

1. Importamos las siguientes librerías:
 - a. Swarm: clase que orquesta varios Crazyflies en paralelo.
 - b. CachedCfFactory: fábrica que crea y gestiona instancias de SyncCrazyflie (necesario para Swarm).
 - c. MotionCommander: interfaz de alto nivel para comandos de vuelo (avanzar, girar, etc.).
 - d. Multiranger: obtiene mediciones de distancia desde el deck Multi-Ranger.
2. Inicializa los parametros de comportamiento:
 - a. URI_LIST: Lista con las direcciones de radio de cada Crazyflie. Debéis sustituirlas por las vuestras (cada dongle y cada dron llevan un ID único).
 - b. THRESHOLD: distancia, en metros, por debajo de la cual consideramos un obstáculo “cercano”.
 - c. FORWARD_SPEED y EVADE_SPEED: controlan qué tan rápido va o se desplaza en lateral.
 - d. LOOP_PERIOD: con qué frecuencia ($\text{Hz} = 1/\text{LOOP_PERIOD}$) lee sensores y envía comandos.
 - e. STATE_FORWARD: modo “avanzar hacia adelante”.
 - f. STATE_EVADE: modo “esquivar” desplazándose en lateral.

3. Declaramos la Función **swarm_obstacle_avoidance**

- a. Argumento scf: es un objeto SyncCrazyflie que ya tiene establecida la conexión radio.
- b. Multiranger(scf): lanza un hilo interno para leer continuamente los cinco sensores de distancia sin bloquear.
- c. with MotionCommander(scf, default_height=0.4)
 - i. Al entrar, sube el dron a 0.4 m;
 - ii. Mientras estemos dentro del with, envía automáticamente mensajes de “mantener posición” (setpoints periódicos);
 - iii. Al salir, aterriza suave y detiene motores.
- d. Máquina de estados
 - i. FORWARD: si $\text{front} \geq \text{THRESHOLD}$, envía velocidad hacia adelante; si $\text{front} < \text{THRESHOLD}$, cambia a EVADE.
 - ii. EVADE: si $\text{front} \leq \text{umbral} \times 1.2$, se mueve lateralmente; en cuanto ve que $\text{front} > \text{umbral} \times 1.2$, vuelve a FORWARD.
- e. Gestión de interrupción
 - i. Capturamos KeyboardInterrupt (Ctrl+C) para que el dron aterrice antes de salir del contexto.

4. Bloque principal:

- a. cflib.crtp.init_drivers(): arranca el driver que gestiona la comunicación radio USB.
- b. CachedCfFactory(): objeto que sabe crear y cachear instancias de SyncCrazyflie para cada URI.
- c. with Swarm(URI_LIST, factory=factory) as swarm:
 - i. Se conectan todos los drones listados en URI_LIST.
 - ii. Al entrar, reserva conexiones; al salir, las cierra.
- d. swarm.parallel(swarm_obstacle_avoidance)
 - i. Ejecuta la función de evasión en cada dron simultáneamente, en hilos independientes.

```
#!/usr/bin/env python3

import logging

import time

import cflib.crtp from cflib.crazyflie.swarm

import Swarm, CachedCfFactory from cflib.utils.multiranger

import Multiranger from cflib.positioning.motion_commander
```

```
import MotionCommander

URI_LIST = [

"radio://0/80/2M/E7E7E7E7E7", # Crazyfly #1

"radio://0/80/2M/E7E7E7E701", # Crazyfly #2

]

THRESHOLD = 0.6 # umbral frontal [m]

FORWARD_SPEED = 0.05 # velocidad avance [m/s]

EVADE_SPEED = 0.1 # velocidad lateral de evasión [m/s]

LOOP_PERIOD = 0.1 # periodo de bucle [s]

STATE_FORWARD = 0 STATE_EVADE = 1

def swarm_obstacle_avoidance(scf):

    """ Lógica de avance–evasión para un solo dron en el enjambre. 'scf' es un
    SyncCrazyfly ya conectado. """

    mr = Multiranger(scf)

    with MotionCommander(scf, default_height=0.4) as mc:
        print(f"[{scf.cf.link_uri}] Despegado, entrando en bucle de evasión")
        state = STATE_FORWARD

        try:
            while True:
                front = mr.front
                if front is None:
                    time.sleep(LOOP_PERIOD)
                    continue

                if state == STATE_FORWARD:
                    if front < THRESHOLD:
                        state = STATE_EVADE
                        mc.stop()
                        print(
                            f"[{scf.cf.link_uri}] Obstáculo a {front:.2f} m → EVADE"
                        )
                    else:
                        mc.start_linear_motion(FORWARD_SPEED, 0, 0)

            else: # STATE_EVADE
```

```
        if front > THRESHOLD * 1.2:
            state = STATE_FORWARD
            mc.stop()
            print(
                f"[{scf.cf.link_uri}] Pared esquivada ({front:.2f} m) → FORWARD"
            )
        else:
            mc.start_linear_motion(0, EVADE_SPEED, 0)

        time.sleep(LOOP_PERIOD)

    except KeyboardInterrupt:
        print(f"[{scf.cf.link_uri}] Interrupción: aterrizando...")

mr.close()
print(f"[{scf.cf.link_uri}] Prueba finalizada")

if name == "main":
    logging.basicConfig(level=logging.ERROR)
    cflib.crtp.init_drivers()
    factory = CachedCfFactory()

    with Swarm(URI_LIST, factory=factory) as swarm:
        swarm.parallel(swarm_obstacle_avoidance)
```

Enunciado

[2 pts] Se tendrá que entregar una memoria explicando todo el proceso realizado e incluir capturas de pantalla y videos para ilustrar dicho procedimiento.

Se pide:

1. Programar una ruta compleja en la que dos Crazyflie se manejen de forma autónoma con un script de python utilizando el posicionamiento Multi-Ranger y la función de Swarm para gestionar varios UAVs al mismo tiempo.